SafeKeep Support Agent

Solution Overview & AI Usage

Sai Varshini Thupakula

Contents

1. Solution overview..... 3
2. How AI was used.....5
3. Example prompts.....6
4. Corrections and refinements..... 7
5. Prompts inside the product.....8
6. Validation summary.....9
7. Known limitations and next steps.....9

1. Solution overview

1.1 What was built

SafeKeep Support Agent is an event-driven AI agent that investigates incidents on an enterprise VM backup platform. When a monitored application reports a problem (high memory, a failed backup job, an unreachable dependency), the Agent opens an investigation, gathers evidence through permitted diagnostic tools, separates confirmed facts from suspected causes, recommends next steps, and proposes corrective actions that run only after a person with the right role approves them. Every step is recorded.

The scenarios are drawn from real experience supporting an enterprise VM backup and restore product: memory growth from stale job sessions, CPU saturation from an authentication loop, full storage pools, failed backup jobs, expired vCenter certificates and storage server crashes. All names and data are fictional.

Table 1. Solution at a glance

Area	Implementation
Applications	Two simulated applications (Backup Server, Backup Storage Server) defined in YAML, with 10 seeded scenarios
AI	Local model through Ollama (qwen2.5:3b) with tool calling; no data leaves the machine
Backend	Java 21, Spring Boot 4.1.2, H2 database
UI	Event simulator, investigation list and detail (live timeline, findings, evidence, approval card, conversation), admin toggles, health and metrics
Quality	145 automated tests, mapped to all 47 requirements

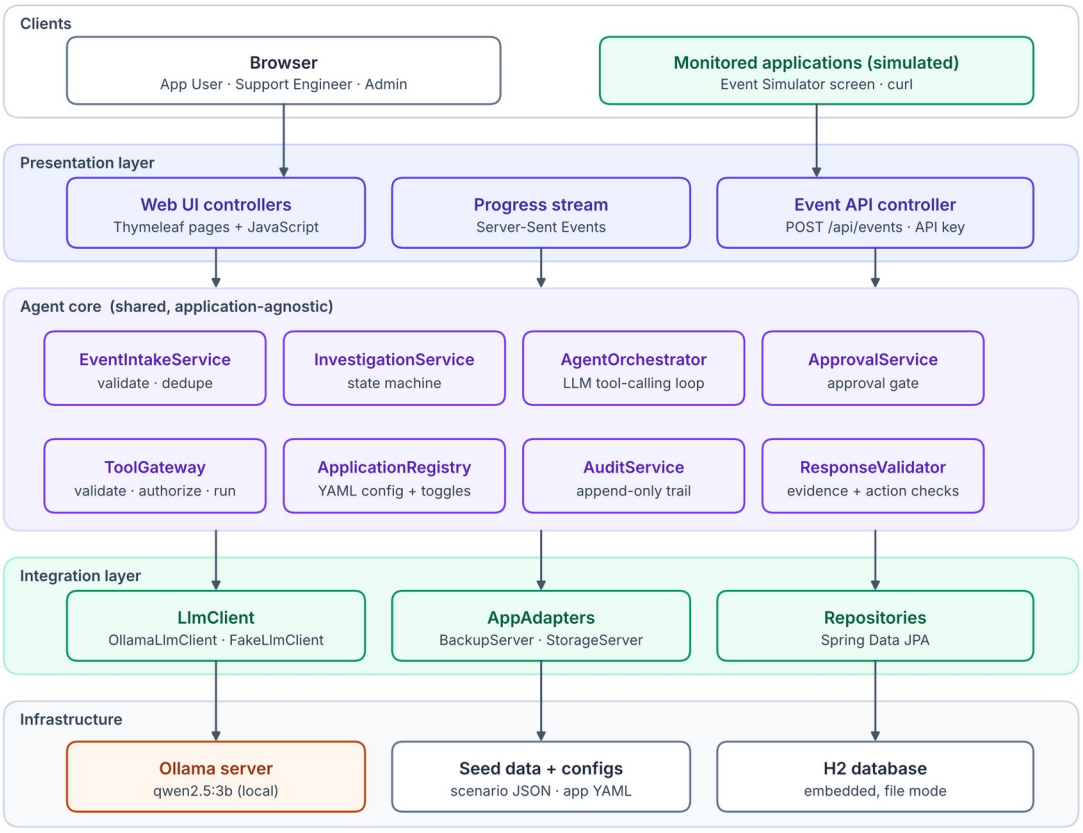


Figure 1. Layered component architecture

1.2 The central design idea

> **The AI reasons; code enforces.** The model only chooses which diagnostic tool to call and writes the analysis. Everything that must be correct is deterministic Java: permissions, the approval gate, state transitions, evidence checks and outcome verification.

Table 2. Guarantees

Guarantee	How it is enforced
The AI cannot change anything	Actions are never offered as callable tools. The only code path that executes an action is the approval service, which checks the user's role.
Suspicions are not shown as facts	A finding labeled CONFIRMED must cite evidence that exists and succeeded; otherwise code downgrades it to SUSPECTED.
Failures are never reported as success	After an action, the system re-reads the application state and evaluates a configured condition before marking anything RESOLVED.
Only permitted data is used	Every tool call passes one gateway: tool exists, is enabled, arguments are valid, runs with a timeout, and is recorded as evidence.

1.3 Deliverables

Table 3. Deliverables

Deliverable	Location
Design: Software Functional Specification	SafeKeep_Functional_Spec.docx
Unit-Test Plan, with requirement-to-test traceability	SafeKeep_Unit_Test_Plan.docx
Implementation	GitHub repository (README explains how to run it)
Solution overview and AI usage	This document

1.4 Running it

```
brew services start ollama && ollama pull qwen2.5:3b
./run.sh                       # starts on http://localhost:8080
./mvnw test                    # 145 tests, no Ollama needed
```

Demo users: alice (application user), sam (support engineer), adam (administrator); password “demo”.

2. How AI was used

2.1 Tools

Table 4. AI tools

Tool	Used for
Claude (AI assistant)	Reading and decomposing the PRD, comparing design options, drafting the specification and test plan, generating code in batches, diagnosing build and test failures
Ollama with qwen3:4b, llama3.2:3b and qwen2.5:3b	Model spikes on the target laptop (M1, 8 GB), and the runtime model inside the product

2.2 Working method

AI was used as a collaborator, not as an answer generator. Each phase followed the same loop: ask the AI to explain or propose options, challenge the result against the PRD and my domain experience, decide, then verify by running something.

Table 5. Division of work

Phase	What the AI did	What I did
Understand the PRD	Decomposed it into 47 numbered requirements; listed 13 ambiguities with proposed decisions	Questioned the scope (agent only or full application? real AI or scripted?), and approved or changed each decision
Ground it in a real domain	Rebuilt the scenarios around backup and restore	Supplied the domain: VM backups, storage servers, data movers, stale sessions, memory leaks, thread contention
Design	Drafted the functional spec, diagrams and test plan	Reviewed structure, length and wording; required clean diagrams and plain language

Phase	What the AI did	What I did
Choose the model	Proposed candidates and a tool-calling test	Ran the spikes on my laptop and compared timings and behavior
Implement	Generated the code in batches	Built and ran it locally, reported compile errors and observed behavior, and drove each fix
Validate	Wrote tests mapped to the plan	Ran the suite and the scenarios in the UI; investigated the failures

3. Example prompts

Representative prompts from the working session, paraphrased. Each one started a short back-and-forth: the AI proposed options or a draft, I pushed back or added domain detail, and we settled on the outcome shown.

Table 6. Example prompts

Area	Prompt (paraphrased)	What we settled on
Scope	“Let's not build the agent in isolation. I want a working product that shows the whole flow: events coming in, the investigation, approval and history. Keep it demo-sized.”	A vertical slice covering every requirement, with an explicit out-of-scope list
Stack	“I'm strongest in Java, so let's use Spring Boot. Keep the UI simple and server-rendered; the focus should be the backend and the agent.”	Spring Boot with Thymeleaf pages and a small script; no frontend toolchain
Requirements	“Break the PRD into numbered requirements so every design section and every test can point back to one.”	47 requirement IDs, used across the spec, code and tests
API design	“Let's fix the API before writing code: one event endpoint where applications authenticate with an API key, and investigation endpoints for questions, approvals and feedback.”	The REST API in Spec Section 11
Users	“We don't need user management. Let's stick to three predefined users, one per role, and spend the effort on enforcing permissions on the server.”	Decision D2: seeded users, role checks on the server
Approvals	“Not every action carries the same risk. Deleting old backups should need an admin; retrying a report shouldn't. Each action should declare its own required role.”	Decision D3: requiredRole and risk per action, in configuration
Domain	“Let's base the scenarios on the VM backup product I worked on: stale job sessions, memory leaks, thread contention, CPU spikes. I can speak to those in depth.”	Scenarios rebuilt around backup and restore
Scenarios	“A failed backup has more than one possible cause, like a full storage pool or an auth loop that opens a new session on every step. Let's have several scenarios per use case, and one where the evidence isn't enough.”	Three job-failure scenarios, including one where the agent must say the cause is unknown
Scenarios	“Let's add a storage server crash, something like a kernel panic from a failing disk.”	Refined together: a crashed server cannot report itself, so one incident produces two events from two applications
Applications	“Let's treat the storage server as its own application with its own events, tools and actions, so the agent clearly works across applications.”	Two applications, each defined only in YAML plus an adapter

Area	Prompt (paraphrased)	What we settled on
Agent design	“Should we have one agent per application? I'd rather keep one shared agent and make the applications pluggable through configuration.”	One orchestrator; application-specific code isolated in adapters (NFR-MAINT-01), proven by a test-only third application
Seed data	“The simulated data should only contain raw metrics and logs, never the answer, so the model actually has to reason.”	Seed data rule A5, enforced by a test that scans every seed file
Model	“Let's keep everything local with Ollama, no cloud models. Try a few small models on my laptop and pick by measured results.”	Three models compared on the same scenario; qwen2.5:3b selected
Agent quality	“The model stopped after one tool call and asked me to check logs it could read itself. Let's handle that in code instead of hoping for a better prompt.”	Code guards: minimum tool calls, duplicate-call skip, use tools before asking
Agent quality	“It found the stale sessions and then left them out of the summary. Let's re-list the evidence before the final answer and make it name the abnormal values first.”	Decision D18: two-phase answer with observations
Safety	“After re-investigating, it proposed a purge when memory was already normal. The system shouldn't propose fixes for problems that aren't present.”	Rule R-V8
Reliability	“What happens if many events arrive at once and all workers are busy? Investigations shouldn't silently get stuck.”	Rejected work is kept and retried every 15 seconds, with a test
Testing	“Let's write the test plan before the code, and give every test its test case ID so the test report maps straight to the plan.”	Test-first plan; IDs in every @DisplayName; 145 tests
Documents	“Keep the documents short and in plain language, and update them after implementation with a revision history.”	Spec and test plan revised to match the code

4. Corrections and refinements

The most useful part of the process was correcting the AI and the model when evidence showed they were wrong. Each row below was found by running something, not by reading.

4.1 Model selection

Table 7. Model comparison

Model	Measured result (scenario S-1A, M1 8 GB)	Decision
qwen3:4b	Correct tool call, but 118 s per call: its reasoning mode could not be disabled	Rejected
llama3.2:3b	1.7 s per call, but stopped after one tool call, ignored evidence it had gathered, and asked the user to check logs it could read itself	Rejected after three prompt refinements
qwen2.5:3b	Two CONFIRMED findings citing the right evidence, the correct action proposed, no unnecessary question	Selected

4.2 Refinements to the agent

Table 8. Agent refinements

Observed problem	Refinement	Kind
The model called get_metrics with {"days": "0"}: a string, and a meaningless value	Argument validation in the tool gateway: safe type coercion, ranges, unknown arguments rejected, errors returned to the model	Code guard
The model stopped after confirming the symptom	If it stops before 4 tool calls, code tells it which tools it has not used (at most twice)	Code guard
The model repeated an identical call	Identical repeats are not executed; the model is pointed to the earlier evidence ID	Code guard
The model asked the user for information a tool could provide	It is sent back once to use its remaining tools before a question is allowed	Code guard
The final answer ignored evidence gathered earlier (1,172 stale sessions)	Two-phase answer: every evidence item is re-listed, and the model must list abnormal observations before its findings	Prompt structure
A re-investigation proposed a fix for a problem that was already gone; verification then passed trivially	Rule R-V8: an action is proposed only when the evidence shows the problem is still present	Validation rule

4.3 Corrections to the AI assistant

Table 9. Corrections

Situation	Correction
The assistant suspected the 4,096-token context window was truncating prompts	Checked the Ollama log: largest request 2,223 tokens, truncated = 0. Hypothesis rejected and no change made.
The plan specified Spring Boot 3	A version check showed 3.5 reached end of life in June 2026; the project uses 4.1.2
curl requests were blocked by CSRF protection	Clients authenticating with an Authorization header are exempt; browser sessions stay protected
A security test expected a redirect but received 401	The application was right: browsers send Accept: text/html. The test was fixed, not the product.
While discussing concurrency, a gap appeared: a full worker queue would leave an investigation stuck	Rejected work is kept and retried every 15 seconds, with a dedicated test
Generated documents were long and used section symbols and automated-sounding phrases	Shortened from 38 to 30 pages and rewritten in plain language

5. Prompts inside the product

The runtime prompts live in PromptBuilder. They evolved with the refinements above.

Table 10. Runtime prompts

Prompt	Purpose	Key content
System	Rules for the investigator	Use only the provided tools; cite evidence IDs; CONFIRMED means measured, SUSPECTED means inferred; never claim an action ran; find the cause, not only the symptom; never ask the user for what a tool can provide
Kickoff	The task	Event type, resource, details, repeat count, recurrence, previous findings and user context
Nudge / use tools first	Guard messages	Lists the tools not used yet
Final instruction	The answer	Evidence re-listed; exact JSON fields; observations first; when to propose an action and when to ask a question
Repair	Invalid output	States the validation error and asks for the JSON again (once)

6. Validation summary

Table 11. Validation summary

Check	Result
Automated tests	145 executed, 0 failures (unit, Spring integration with a scripted model, MockMvc security)
Requirement coverage	All 47 requirements traced to at least one test (Unit-Test Plan, Section 6)
End-to-end with the real model (S-1A)	Stale sessions and memory reported as CONFIRMED with correct evidence; purge proposed; support engineer approval executed the action; verification re-read memory (61%) and set RESOLVED
Role enforcement	Application user's approval attempt rejected (403, “Requires SUPPORT_ENGINEER approval”)
Admin toggle	With get_logs disabled, the agent never used it on a new run
Re-investigation with context	New tool calls and an updated analysis; led to rule R-V8

7. Known limitations and next steps

Table 12. Limitations

Limitation	Next step
Simulated applications	Replace adapters with HTTP integrations; the agent core is unchanged
A 3B model on a laptop: about 2–4 minutes per investigation	A larger model on a GPU host; the model is a configuration setting
Admin toggles are not persisted	Store them in the database
Three seeded users	SSO with roles from directory groups
Correlation is per event type	Time-window clustering of related events across types